PROTOTYPE D — PRECISION WASHING

Roblox Minimum Functional Fantasy & Technology Spike | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-D
Primary fantasy: "Wherever I aim the washer, I can remove grime and reveal a clean surface."
Reference products (inspiration only): *Pressure Wash Simulator*, *Wash the Car*
First-pass effort assumption: 6–10+ focused developer-days (6 hours/day), **conditional on a 1–2 day technical spike**. Duration is not verified competitor production time.
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. Separate from *Thaw a Creature*.

1. Research decision — more fidelity, more risk

Does precise aiming and progressive surface cleaning deliver enough additional tactile satisfaction to justify its substantially greater engineering and asset setup cost compared with discrete clearing and reveal prototypes?

- **Hypothesis D-H1:** Aimed cleaning feels responsive and consistent enough that players enjoy the stroke itself, not just the finished state.
- **Hypothesis D-H2:** Once an object uses a cleanable surface scheme, new similar objects can reuse the technology at an acceptable content-authoring cost.
- **Disconfirmation:** The spray feels inconsistent, patch boundaries look artificial, aiming on mobile is frustrating, or new 3D objects require major per-object engineering.
- **Do not claim:** A tiled mock-up proves the feasibility of continuous paint-like surface removal over arbitrary curved meshes. The P0 prototype is deliberately an **approximation**.

2. Mandatory early technology gate (D-G0)

Time-box: 1–2 developer-days, included in the 6–10+ total estimate. Before implementing a full round:

1. Make one boxy vehicle side panel with a **6 x 4 local dirt-patch grid** (24 patches).
2. Raycast a washer stream at the panel; hitting the targeted patch changes it locally in two stages.
3. Test aiming with desktop mouse and a narrow mobile viewport.
4. Check whether cleaning reliably occurs at the place indicated by the spray and whether patch transitions look tolerable.
5. Measure actual frame time, bad targeting, client-server correction, and how long adding a second cleanable side panel takes.

Continue only if: The tester can deliberately choose which patch to clean, aim corresponds to the observed effect, and mobile use is workable without special instruction. If this is not achieved within the time box, **stop D** rather than building a car wash economy to compensate. A different technology approach requires a separate decision and revised budget.

3. P0 scope after the technical gate

Component	Minimum functionality	Exclusions
Vehicle	One stationary, low-poly, flat-panel van / boxy car	Driving, multiple vehicles, curved sports cars
Washable faces	Three rectangular cleanable faces (side, front, rear)	Arbitrary curved meshes; full texture UV painting
Dirt patches	72 total anchored surface patches: 24 per face	High-resolution per-pixel grime
Patch state	Two accepted spray passes per patch	Dirt thickness variations / chemical systems
Tool	One washer with directed beam + simple mist	Multiple nozzles, tanks, pressure upgrades
Input	Hold spray, aim at specific visible dirty patch	Gesture-only precision brush editor
Progress	144 accepted patch hits target 100%	Cash, tips, rating, automation
Completion	Clean van visibly transformed; replay/finish choices	Car collections, garages, sales
Replay	Reset dirt on identical van	New vehicle progression, upgrades
Server	Exact patch hit acceptance and progress authoritative	Client-controlled dirt percentage

First-pass tuning: three 6x4 grids; two hits/patch; action acceptance <=6.5 hits/sec; target <=13 studs; approximate 45–90 sec first round. Measure actual first-round time; do not assume 144 actions guarantees the target duration.

Scope honesty: This tests *spatial targeting on multiple 3D faces* with tiled dirt. It will not produce the smooth irregular streaks of a polished PC PowerWash-style simulation. Those require separate art/UV/rendering R&D and are not authorized by this P0 document.

4. Player-facing experience

Entry: One very dirty, stationary boxy van sits near the spawn. The clean/dirty contrast is immediately visible. A simple HUD says **"Wash the van — aim and hold Spray"**, with percentage progress.

Spray: The player aims the tool and activates Spray. A thin stream/tracer indicates direction; the targeted dirt patch becomes partially cleaned and then disappears after the required accepted samples. A short wet impact effect and low-intensity sound follow valid accepted hits. Hitting a clean patch, sky, ground or wall makes no progress.

Traversal: Player walks around the van to reach each face, rather than washing the whole vehicle from one angle. Keep enough space around the three exposed faces and avoid a roof panel that requires awkward jumping or camera clipping.

Finish: All targeted faces read clean. Display `100% CLEAN`, a short before/after camera framing, and neutral `Try another round` / `Finish` choices.

Mobile: A large `Spray` button is separate from camera movement. Target proxies are slightly more forgiving than their visual geometry, but cleaning still occurs where the user aims. If a phone player cannot target the top/edge patches, the test fails rather than silently using an area-wide cleaning mechanic.

5. Art, layout and technical asset brief

Item	Qty	Notes
Blockout van	1	Flat cleanable faces, readable silhouette
Washable face proxies	3	Side/front/rear; consistent local grid basis
Dirt patches	72	Visually flush to panels; queryable targets, no physical collision
Washer	1	Reticle, short stream, hit marker; no fluid simulation
Ground/test bay	1	Enough room for all three faces, neutral lighting
HUD / completion	1	Progress and touch-friendly finish/replay
Spray feedback	1 shared lightweight visual	Pool and cap effects; no persistent water particles
Alternate dirt layout	1 seeded reset	No need for a second vehicle

Important rendering rule: Dirt patches should not visibly float, flicker against the vehicle, or create collision barriers. Set up stable offsets relative to their cleanable face to avoid z-fighting; test in Studio under different camera distances. Use one material scheme and a clear half-clean state, not hundreds of unique textures.

6. State and networking model

Round data

`RoundState = { RoundId, PatchRemainingHits[1..72], AcceptedHits, TotalTargetHits=144, State, StartedAt }`. `Progress01 = AcceptedHits / 144` server-side.

The server is the only authority that decides which patch was hit and whether it counts. The client sends `SprayRequest(RoundId, PatchId, optionalAimRay)` as a **hint**, not a result. To avoid trusting arbitrary client `PatchId`, the server independently validates the candidate by geometry and reach. For first-pass P0, the server can calculate a line from a known valid player/head/tool region to the expected patch target and check obstruction and facing, rather than attempting to replicate exact camera input perfectly.

Minimum server guard contract:

1. Check live, active player and correct RoundId; no spray during replay/reset.
2. Verify patch exists in the active vehicle and is not already clean.
3. Require <=13 studs between legitimate character origin and patch; target is in a reasonable forward-facing cone and not obscured by another world surface.
4. Rate limit accepted patch hits to <=6.5 per second (config first pass). Reject duplicates, forged target IDs and stale packets.
5. Apply exactly one dirt-state step, then replicate accepted patch visual state and total progress.
6. Complete once at exactly 144 successful state changes; stop accepting action requests before replay.

Aim validation limitation: A server cannot trivially know the exact local player's camera ray at the exact input time. A client-supplied ray may help the server reconcile targeting, but must be normalized, range-bounded and checked against the authoritative world. Do not claim mathematically identical client/server aiming or use untrusted client hit results as proof.

Do not stream a RemoteEvent per render frame. Send a bounded number of requested spray samples while the input is held (e.g., 5–7 requests/s) and lerp/trail cosmetics locally. The server's accepted-hit sampling governs progress. For a noncompetitive prototype, brief local visual anticipation is allowed if reconciliation is visible and not misleading.

7. Optional fidelity branch — not P0

Roblox `EditableImage` provides runtime image editing but has memory budgets and security/permission restrictions, including a documented maximum image resolution and asset ownership requirements. A future image-based cleaning system would require robust UV or panel-coordinate mapping, appropriate performance tests and a new asset pipeline. Roblox Creator Hub documents those constraints; its availability does not make a production-ready texture-washing solution automatic.

Option D1 (P0): 72-part tiled dirt approximation (approved for *testing if implemented*).
Option D2 (unapproved future R&D): Fine-grained image-mask clearing for rectangular surfaces.
Option D3 (unapproved future R&D): Arbitrary curved-object UV/grime simulation.

Do not silently replace D1 with D2 or D3; they change both scope and risk.

8. Small implementation architecture

```
ReplicatedStorage/RestD/Config.lua
ReplicatedStorage/RestD/Remotes/SprayRequest
ReplicatedStorage/RestD/Remotes/PatchStateChanged
ReplicatedStorage/RestD/Remotes/RoundStateChanged
ServerScriptService/RestD/RoundService.lua
ServerScriptService/RestD/SprayValidationService.lua
ServerScriptService/RestD/SurfaceStateService.lua
StarterPlayer/StarterPlayerScripts/RestDClient.lua
Workspace/RestD_Arena/Van/CleanableFaces/*
```

Keep each patch anchored, stable and identified by one server-defined `PatchId`. All counters derive from server state. Client modules own camera reticle, stream VFX and UI only.

9. Production sequence with explicit exit gates

Step	Work	PASS to continue
D-00	Technology spike: one 24-patch side face	Accurate and satisfying local aim desktop + mobile; no serious stutter
D-01	Three cleanable faces; raycast targeting	Every patch can be reached without awkward movement
D-02	Server action guards and 2-hit patch state	No false hit, double progress, or remote spam bypass
D-03	Spray cosmetics, completion and replay	0–100% credible visual transformation and fresh reset
D-04	Mobile refinement, frame-time check, logs	Touch aiming succeeds; performance reasonable on available test device
D-05	Full acceptance pass, actual effort accounting	Decide whether fidelity is worth expanding

Cut order: decorative steam/water VFX -> complex tool model -> before/after camera animation. Do **not cut:** precise localized targeting, stable visual response, mobile aim, two-state surface transitions, server authority, completion/replay.

10. Acceptance suite (D-T01 to D-T16)

ID	Test	PASS condition
D-T01	Technical spike	24-patch face passes desktop aim and mobile aim, within time box
D-T02	Shot targeting	Only target patch responds; nearby patches stay unchanged
D-T03	Partial clean	First accepted hit visibly changes only that patch
D-T04	Clean result	Second hit fully removes patch and exposes clean face
D-T05	Missed ray	No server progress for sky, ground, clean area, out-of-range area
D-T06	Side/front/rear	All 72 patch IDs accessible and correctly oriented
D-T07	Camera edge tests	No erratic selection near corners or roofline
D-T08	Distant/occluded request	Server rejects forged or unreachable patch IDs
D-T09	Spam test	Maximum accepted rate enforced, no free progress
D-T10	Concurrent/stale packets	Exact one patch state transition per accepted hit; old round ignored
D-T11	Input hold/release	Spray starts/stops reliably; no stuck firing after respawn
D-T12	Completion	144 valid hits -> exactly one 100% event
D-T13	Replay	Dirt and progress reset, new RoundId, no old effects
D-T14	Mobile	Every face can be washed without hidden controls or pinching
D-T15	Performance	Real device or emulated narrow viewport tested; no persistent memory growth
D-T16	Scope honesty	Video of P0 shows tiled fidelity; no claim of smooth continuous washing

11. Decision and measurement

Alongside the shared protocol, record:

- Proportion of spray attempts whose **visual hit location matches player intent** (observation + tester question).
- Number of missed/incorrect target patches and time needed for the final 10%.
- Whether a user reports pleasure in precise strokes, or only in the final before/after.
- Engineering hours spent on the surface mapping/aim problem *alone*.
- Minutes to author and verify a second cleanable flat panel.
- Client FPS, device model/class, remote rejection counts, and reset defects.

Proceed beyond P0 only if: the precise interaction feels clearly better than coarse cleaning to testers, mobile controls hold up, and actual cost is defensible. **Stop if:** players mostly respond to the final clean state, because prototype A or C may supply that payoff much more cheaply.

12. Risks and AI guardrails

Risk	First response
Appearance too obviously tiled	Record as failure of fidelity; do not silently add custom shaders

Risk	First response
Spray doesn't match aim	Correct query proxies and authority/preview sync before visual polish
EditableImage temptation	Not approved in P0; separate R&D spike and permission/performance audit
Fine aim fails on phone	Widen query targets slightly while preserving local response; retest
Surface edges flicker	Offset surface patches and stabilize per-face alignment
New objects require manual mapping	Log authoring minutes; compare honestly with A/B/C

Instructions to Claude / Codex:

1. Begin with **D-G0 technical spike only**; do not build the full van before its result is approved.
2. Do not substitute a generic one-click wash, entire-face deletion, or global progress counter for localized washing.
3. Do not add vehicle driving, inventories, equipment upgrades, currency, a garage economy, robots, or collectible cars.
4. Keep patch state and progress server-authoritative. Consider a client ray only a validated hint.
5. Do not promise PowerWash-like continuous dirt removal from tile-based P0. Explicitly describe fidelity limitations.
6. Do not touch *Thaw a Creature*. Preserve all unrelated Roblox places and teammate code.
7. Report each acceptance test as PASS, FAIL or NOT TESTED based on actual Studio/client observation; never claim work was run when it was only designed.

13. Technical references

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService
- https://create.roblox.com/docs/reference/engine/classes/EditableImage
- https://create.roblox.com/docs/reference/engine/classes/CollectionService